EXPERIMENT – 01

AIM – WAP to implement bubble sort.

. **PROGRAM :-**

C bubbleSort.c > main()

```c
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;

        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                swapped = 1;
            }
        }

        if (swapped == 0) {
            break;
        }
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    bubbleSort(arr, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

OUTPUT :-

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Vatsal\Vatsal.c> cd "c:\Users\Vatsal\Vatsal.c\" ;
Enter number of elements: 4
Enter 4 elements: 4
8
1
5
Sorted array: 1 4 5 8
PS C:\Users\Vatsal\Vatsal.c>
```

EXPERIMENT – 02

AIM – WAP to implement Selection sort.

. **PROGRAM :-**

C selectionSort.c > main()

```c
#include <stdio.h>

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Original array: ");
    printArray(arr, n);

    selectionSort(arr, n);

    printf("Sorted array: ");
    printArray(arr, n);

    return 0;
}
```

OUTPUT :-

```
PS C:\Users\Vatsal\Vatsal.c> cd "c:\Users\Vatsal\Vatsal.c\"
Enter the number of elements: 4
Enter 4 elements:
2
1
4
7
Original array: 2 1 4 7
Sorted array: 1 2 4 7
PS C:\Users\Vatsal\Vatsal.c> cd "c:\Users\Vatsal\Vatsal.c\"
```

EXPERIMENT – 03

AIM – WAP to implement Insertion sort.

. **PROGRAM :-**

C insertionSort.c > main()

```c
#include <stdio.h>

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    insertionSort(arr, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

OUTPUT :-

```
PS C:\Users\Vatsal\Vatsal.c> cd "c:\Users\Vatsal\Vatsal.c\"
Enter number of elements: 4
Enter 4 elements: 2
3
6
1
Sorted array: 1 2 3 6
PS C:\Users\Vatsal\Vatsal.c>
```

EXPERIMENT – 04

AIM – WAP to implement Linear Search

. **PROGRAM :-**

C linearSearch.c > ...

```c
#include <stdio.h>

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;  // return index if found
        }
    }
    return -1;  // not found
}

int main() {
    int n, key;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    int result = linearSearch(arr, n, key);

    if (result != -1) {
        printf("Element found at index %d (position %d)\n", result, result + 1);
    } else {
        printf("Element not found in the array\n");
    }

    return 0;
}
```

OUTPUT :-

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Vatsal\Vatsal.c> cd "c:\Users\Vatsal\Vatsal.c\" ;
Enter number of elements: 5
Enter 5 elements: 2
3
5
9
4
Enter element to search: 5
Element found at index 2 (position 3)
PS C:\Users\Vatsal\Vatsal.c>
```

EXPERIMENT 05

AIM – WAP to implement Binary Search

. **PROGRAM :-**

```c
C binarySearch.c > main()
#include <stdio.h>

int binarySearch(int arr[], int n, int key) {
    int left = 0, right = n - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == key) {
            return mid;
        }
        else if (arr[mid] < key) {
            left = mid + 1;
        }
        else {
            right = mid - 1;
        }
    }

    return -1; // not found
}

int main() {
    int n, key;
    printf("Enter number of elements (sorted array): ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d sorted elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    int result = binarySearch(arr, n, key);

    if (result != -1) {
        printf("Element found at index %d (position %d)\n", result, result + 1);
    } else {
        printf("Element not found in the array\n");
    }

    return 0;
}
```

OUTPUT :-

```
PS C:\Users\Vatsal\Vatsal.c> cd "c:\Users\Vatsal\Vatsal.c\" ;
Enter number of elements (sorted array): 5
Enter 5 sorted elements: 1
3
5
7
9
Enter element to search: 7
Element found at index 3 (position 4)
PS C:\Users\Vatsal\Vatsal.c>
```